

ShopEase

Cloud Infrastructure Planning and Strategy

Week 1 Task: Cloud Infrastructure Planning and Strategy Simulation

Project Type	Fictional E-Commerce Cloud Architecture
Cloud Provider	Amazon Web Services (AWS)
Primary Goal	Scalable, secure, highly available and cost-conscious infrastructure
Document Includes	Architecture diagrams, detailed plan, cloud-service justification, cost strategy, risk assessment and assumptions

Planning scope: This report focuses on cloud architecture and engineering decisions rather than a full production deployment.

1. Executive Summary

This report proposes an AWS cloud infrastructure for **ShopEase**, a fictional e-commerce platform. The design supports normal traffic and sudden increases during sales or festival periods. The architecture emphasizes scalability, availability, security, disaster recovery, monitoring and cost control.

The solution uses Amazon Route 53 for DNS, CloudFront for content delivery, an Application Load Balancer for request distribution, Amazon EC2 for application compute, Auto Scaling for elastic capacity, Amazon RDS for the relational database, Amazon S3 for object storage, Amazon VPC for network isolation, IAM and Security Groups for access control, CloudWatch for monitoring, and AWS Backup/RDS backup capabilities for recovery.

The design is intentionally practical for a medium-sized online retail platform. It can later be extended with services such as AWS WAF, ElastiCache, managed containers or serverless components when measured requirements justify them.

2. Project Scenario and Objectives

ShopEase allows customers to register, sign in, browse products, view images, maintain a shopping cart and place orders. Transactional data such as customer, product and order records is stored in a relational database, while product images and static objects are stored separately.

- Design infrastructure that remains responsive when traffic changes.
- Reduce the impact of individual server or Availability Zone failures.
- Protect application and database resources from unnecessary public exposure.
- Provide monitoring, backups and a clear recovery approach.
- Use cloud elasticity and managed services to control operational effort and cost.
- Document assumptions, risks and reasons for selecting each AWS service.

3. Requirements Analysis

3.1 Functional Requirements

- Customer registration and login.
- Product browsing, searching and product-detail viewing.
- Shopping cart operations and order placement.
- Storage of customer, product and order records.
- Delivery of product images and other static content.
- Support for multiple concurrent users.

3.2 Non-Functional Requirements

- High availability and fault tolerance.
- Horizontal scalability during demand spikes.
- Secure network and access-control design.
- Acceptable response time for web users.
- Monitoring and alerting for operational visibility.
- Backup and recovery for important data.
- Cost optimization without sacrificing core reliability.

4. Proposed AWS Architecture

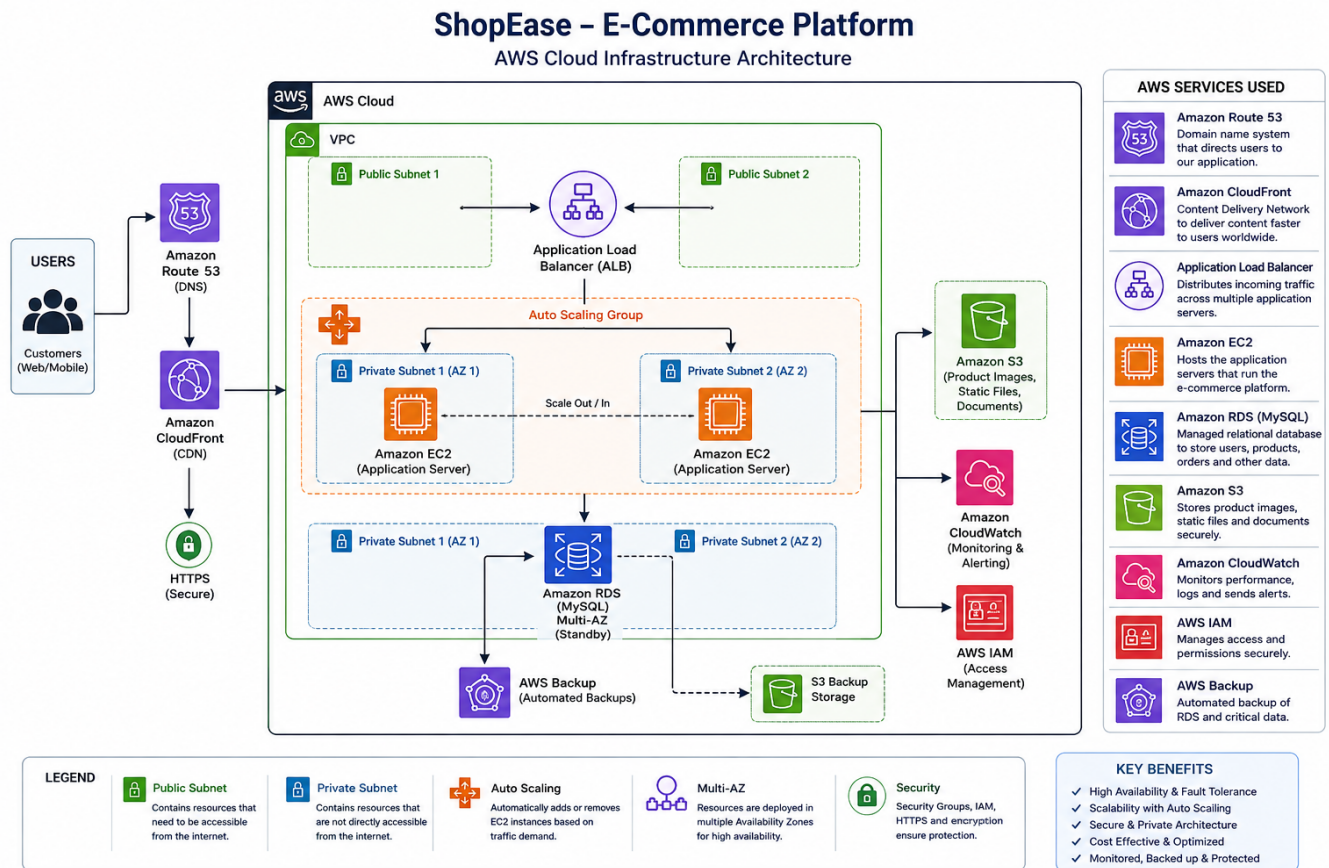


Figure 1. Proposed ShopEase AWS cloud infrastructure architecture.

Customer requests are resolved through Route 53 and accelerated through CloudFront. HTTPS protects traffic in transit. The Application Load Balancer distributes requests to EC2 application instances in an Auto Scaling group. EC2 accesses RDS for transactional data and S3 for product images/static objects. CloudWatch provides monitoring and alerting, while IAM and network controls restrict access.

4.1 Request Flow

Step	Flow	Purpose
1	Customer -> Route 53	Resolve the ShopEase domain to the correct cloud endpoint.
2	Route 53 -> CloudFront	Serve cached content closer to users and reduce repeated origin traffic.
3	CloudFront -> Load Balancer	Forward dynamic application requests to the application tier.
4	Load Balancer -> EC2	Distribute requests across healthy application instances.
5	EC2 -> RDS / S3	Read or write transactional data and retrieve or store objects.
6	Resources -> CloudWatch	Collect operational metrics, logs and alarms.

5. Cloud Service Selection and Justification

Service	Role	Why Selected
Amazon Route 53	DNS	Managed DNS that can reliably direct users to AWS endpoints.
Amazon CloudFront	CDN	Caches static content closer to users, improves delivery speed and reduces repeated origin workload.
Application Load Balancer	Traffic distribution	Routes HTTP/HTTPS traffic to healthy application instances and avoids dependence on one server.

Service	Role	Why Selected
Amazon EC2	Application compute	Flexible virtual servers suitable for a conventional web application and compatible with Auto Scaling.
EC2 Auto Scaling	Elastic capacity	Adds or removes instances according to demand, supporting performance and cost control.
Amazon RDS for MySQL	Relational database	Managed relational database suitable for structured customers, products and orders while reducing administration.
Amazon S3	Object storage	Durable and scalable storage for product images, static files and selected backup objects.
Amazon VPC	Network isolation	Provides logical network boundaries, subnet design and controlled traffic paths.
AWS IAM	Identity and access	Supports least-privilege permissions for users, roles and services.
Security Groups	Network firewall	Restricts inbound and outbound traffic at resource level.
Amazon CloudWatch	Monitoring	Central service for metrics, logs and alarms.
AWS Backup / RDS backups	Recovery	Automates backup policies and supports restoration after data loss or failure.

6. Detailed Infrastructure Plan

6.1 Network Design

A dedicated VPC is used as the network boundary. The load-balancing layer is reachable from the internet, while application and database resources are placed in private network segments where practical. The database is not intended to accept direct public internet connections. Security Groups allow only the traffic required between tiers.

6.2 Compute and Auto Scaling

The application runs on multiple EC2 instances rather than one large server. An Auto Scaling group maintains healthy capacity and can add instances when demand increases. Scaling policies may use CPU utilization, request count or other application metrics. When demand falls, excess capacity can be removed to avoid paying for idle compute.

6.3 Load Balancing and Availability

The Application Load Balancer receives application traffic and sends it only to healthy targets. Application instances are spread across more than one Availability Zone so that failure of one instance or zone does not automatically make the whole application unavailable.

6.4 Database

Amazon RDS for MySQL stores structured transactional data. A Multi-AZ configuration is proposed for higher availability, with automated backups and a defined retention policy. The application connects to the database through controlled private network access.

6.5 Storage and Content Delivery

Amazon S3 stores product images and static objects rather than keeping them on EC2 disks. CloudFront caches suitable static content, reducing latency and repeated origin traffic. S3 lifecycle rules can later move older objects to lower-cost storage classes where appropriate.

6.6 Monitoring and Operations

CloudWatch is used for infrastructure metrics, logs and alarms. Key observations include EC2 health and CPU, load balancer request/error patterns, database utilization and application errors. Monitoring data also supports capacity and cost decisions.

6.7 Backup and Disaster Recovery

RDS automated backups and snapshots protect database data. Important objects are stored in durable S3 storage and backup policies are defined according to retention requirements. The application tier is designed to be replaceable through Auto Scaling rather than relying on manual recovery of a failed EC2 instance.

6.8 Security Plan

- Use HTTPS for customer-facing traffic.
- Apply IAM least-privilege permissions and prefer roles over long-lived credentials.
- Use Security Groups to allow only necessary communication between the load balancer, application and database tiers.
- Keep the database private and prohibit direct public database access.
- Encrypt sensitive data in transit and at rest where supported.
- Enable logging and monitoring for suspicious or abnormal activity.
- Keep operating systems and application dependencies patched.

7. Cost Estimation Strategy

This planning task uses a **cost strategy** rather than claiming a fixed monthly bill because actual AWS charges depend on region, traffic, data transfer, instance sizes, storage, database configuration and usage duration. Before production deployment, the architecture should be entered into the AWS Pricing Calculator using expected traffic and capacity.

Area	Cost Strategy	Reason
EC2	Start with right-sized instances and use Auto Scaling.	Avoid continuously running unnecessary compute capacity.
Load Balancer	Use one application load-balancing layer per required environment.	Provides availability without duplicating unnecessary components.
RDS	Choose instance and storage size from expected transactions; monitor and resize.	Database over-provisioning can become a major fixed cost.
S3	Store images/static objects in S3 and apply lifecycle rules where appropriate.	Object storage is more suitable than application-server disks for these files.
CloudFront	Cache frequently requested static content.	Reduces repeated origin requests and improves user performance.
Backups	Set retention based on business recovery needs.	Unlimited retention creates avoidable storage cost.
Monitoring	Create useful alarms and review metrics regularly.	Identifies idle or oversized resources and abnormal consumption.

Estimation method: define expected monthly users and requests -> estimate EC2 capacity and operating hours -> estimate RDS size/storage -> estimate S3 storage and requests -> estimate outbound data transfer and CloudFront usage -> include load balancing, monitoring and backup usage -> review the total and perform a demand sensitivity check.

8. Risk Assessment Analysis

Risk	Likelihood	Impact	Mitigation
Sudden traffic spike	High	High	Application Load Balancer, Auto Scaling, CloudFront caching and capacity alarms.
EC2 instance failure	Medium	Medium	Health checks and Auto Scaling replacement across multiple Availability Zones.
Database outage	Low-Medium	High	RDS Multi-AZ, automated backups, monitoring and tested restore procedures.
Accidental data loss	Medium	High	Backups/snapshots, retention policies and restricted delete permissions.
Unauthorized access	Medium	High	IAM least privilege, roles, Security Groups, HTTPS, encryption and access reviews.
Application vulnerability	Medium	High	Patch management, secure development, dependency updates, logging and security testing.
Availability Zone disruption	Low	High	Distribute application resources across multiple Availability Zones and use Multi-AZ database design.
Unexpected cloud bill	Medium	Medium-High	Right-sizing, Auto Scaling limits, budgets/alerts, monitoring and periodic cost reviews.
Backup cannot restore	Low-Medium	High	Perform scheduled recovery tests and document recovery procedures.
Configuration error	Medium	Medium-High	Change review, documented configurations and preferably infrastructure-as-code in later phases.

9. Assumptions

- ShopEase is a fictional medium-sized e-commerce platform used for infrastructure planning.
- AWS is the selected cloud provider for the simulation.
- The application uses a conventional web/application-server model that can run on EC2.
- MySQL is suitable for the structured transactional data used by ShopEase.
- Traffic varies and can increase sharply during promotions and festival sales.
- The service is expected to be available continuously, making redundancy important.
- Payment processing would normally use a compliant external payment gateway; raw card details are not assumed to be stored by ShopEase.
- Exact traffic, storage and transaction volumes are not provided; therefore exact monthly pricing is intentionally not claimed.
- The design is a production-oriented baseline, but final sizing requires load testing and real usage data.
- Additional services such as AWS WAF, ElastiCache, containers or serverless components can be evaluated later if requirements justify them.

10. Implementation and Validation Plan

Phase	Activity	Output
1. Requirements	Confirm traffic, availability, security, storage and recovery needs.	Approved requirements and assumptions.
2. Architecture	Create VPC, subnet, compute, database, storage and traffic-flow design.	Architecture diagram and service list.
3. Security	Define IAM roles, Security Groups, HTTPS and private database access.	Security controls matrix.
4. Resilience	Define Multi-AZ, Auto Scaling, health checks and backups.	Availability and recovery plan.
5. Cost	Estimate usage with realistic workload assumptions.	Pricing estimate and optimization actions.

Phase	Activity	Output
6. Testing	Perform load, failure, restore and security tests in a non-production environment.	Validation evidence and corrections.
7. Review	Compare results against requirements and refine sizing.	Final approved design.

11. Conclusion

The proposed ShopEase architecture provides a balanced cloud foundation for an e-commerce application. Load balancing and Auto Scaling address variable demand; Multi-AZ design and backups improve resilience; VPC isolation, IAM, Security Groups and HTTPS strengthen security; S3 and CloudFront support efficient static-content delivery; and CloudWatch provides operational visibility. The cost strategy focuses on right-sizing and elasticity instead of permanent over-provisioning.

Overall, the plan satisfies the Week 1 objective by documenting the proposed architecture, service-selection rationale, scalability approach, security and disaster-recovery considerations, cost-estimation strategy, assumptions and risk mitigations.

Appendix A - Submission Checklist

- PDF contains the architecture diagram.
- Detailed written infrastructure plan is included.
- Selected AWS services are justified.
- Load balancing and Auto Scaling are explained.
- Security, monitoring and disaster recovery are addressed.
- Cost estimation strategy is documented.
- Risk assessment and mitigation table is included.
- Planning assumptions are explicitly stated.